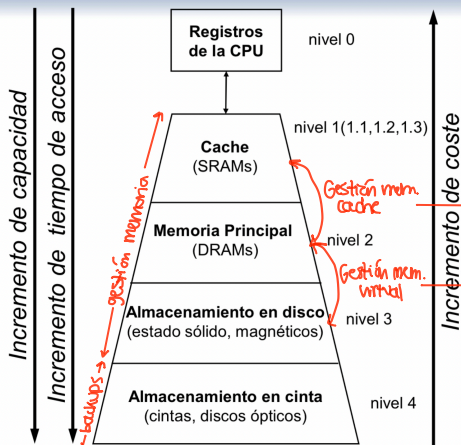


Tema 2. Segmentación

JERARQUÍA DE MEMORIA



Objetivos de la gestión de la jerarquía de la memoria

- Optimizar uso de la memoria

↳ Tiempo de acceso similar al del sistema más rápido

↳ Coste por bit similar al del sistema más barato

→ mediante Hardware (MMU)

→ mediante Hardware (MMU) y Sistema Operativo

↑ tamaño memoria ↑ rapidez ↓ coste

Propiedades de la jerarquía de memoria

- Inclusión: cualquier info almacenada en M_i debe encontrarse también en

$M_{i+1}, M_{i+2}, \dots, M_n$ $M_1 \subset M_2 \subset \dots \subset M_n$

- Coherencia: copias de la misma info en distintos niveles deben actualizarse cada vez que se modifica M_i en $M_{i+1}, M_{i+2}, \dots, M_n$.

- Localidad: referencia a memoria para acceso a datos / inst. están agrupados en ciertas regiones del tiempo y espacio

Bloque unidad mínima de transferencia entre dos niveles

- En cache → línea
- En mem. virtual → página o segmento

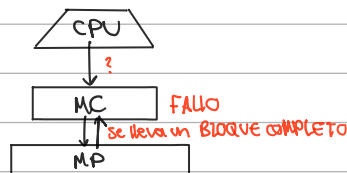
Acierto (hit) el dato solicitado está en el nivel i

- Tasa de aciertos (hit ratio) fracción de accesos acertados
- Tiempo de acierto (hit time) tiempo de detección del acierto + tiempo de acceso

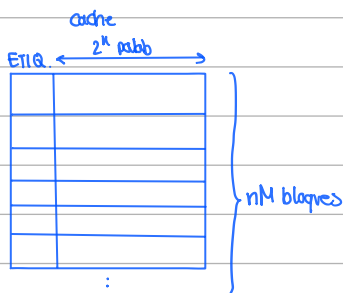
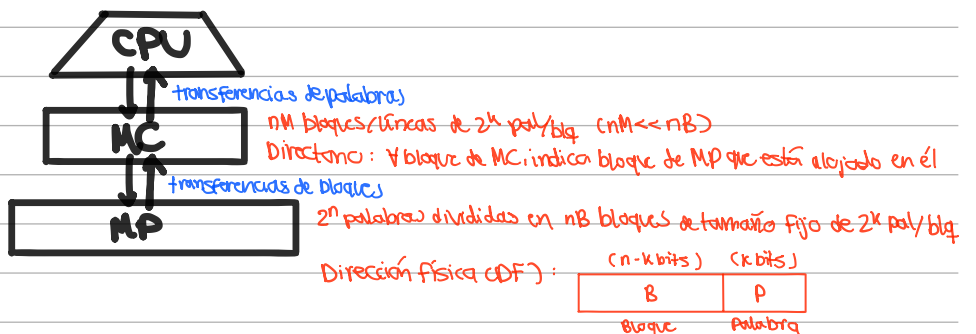
Fallo (miss) el dato solicitado no está en nivel i y es necesario buscarlo en el nivel $i+1$

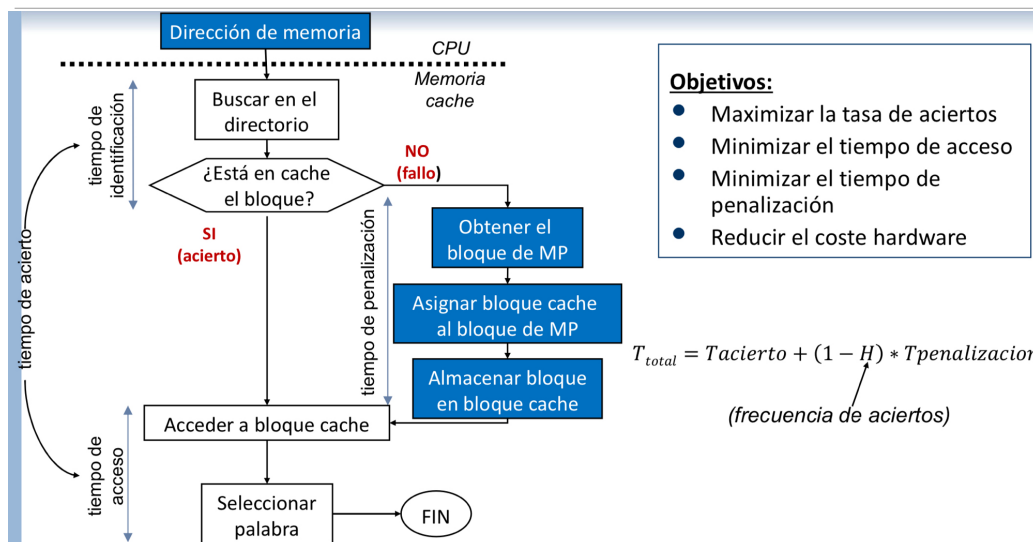
- Tasa de fallos (miss ratio) fracción de accesos fallados o $1 - \text{tasa de acierto}$
- Tiempo de penalización por fallo (miss penalty) tiempo invertido para mover un bloque al nivel $i+1$ al i cuando hay fallo

* Req. Tiempo acierto $\ll$ penaliz. de fallo



Gestión de la memoria cache

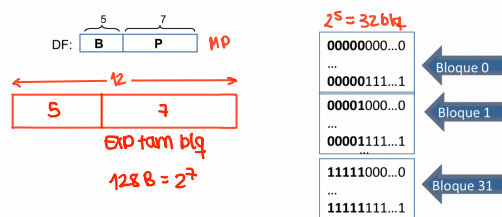




¿Cómo sabemos que un dato está en MC?, si está, ¿cómo lo encontramos?

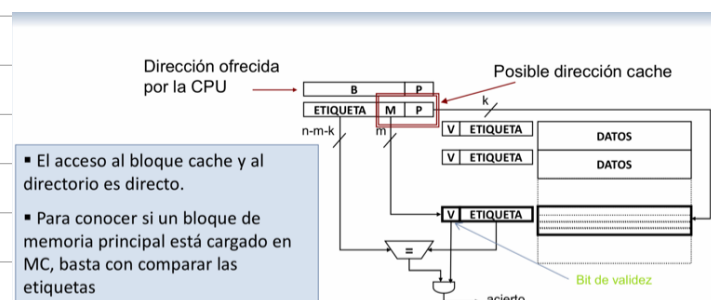
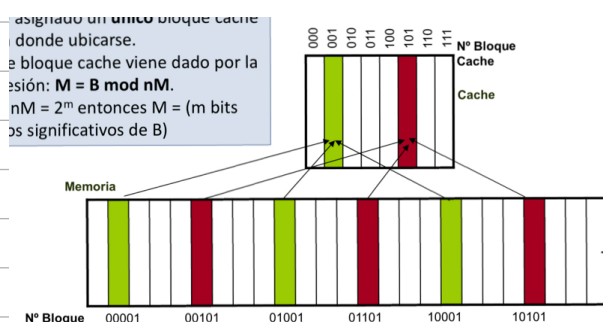
Política de emplatamiento determina en qué bloques de MC puede cargarse el bloque de MP

- Tamaño de bloque 128 palabras $\Rightarrow k=7$ 2^7 pal / bloque
- Memoria cache con 8 bloques
- Memoria principal 4k palabras $\Rightarrow n = 12$ 2^{12} pal aprox. / bloque



-Emplat. directo

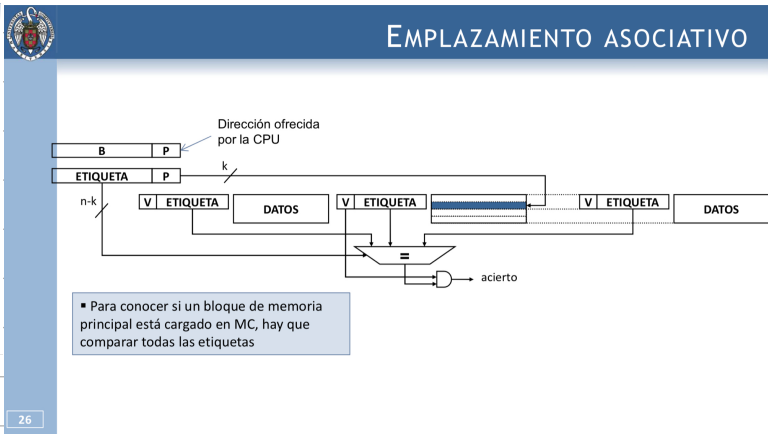
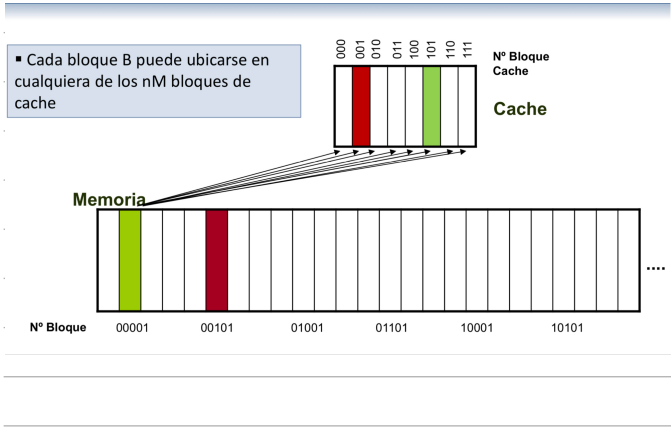
cada bloque B de memoria principal tiene asignado un único bloque cache M en donde ubicarse ($M = B \bmod nM$)



¿Cuánto bits tiene m para los datos del ejemplo?
 $m = \text{bits bloque de la MP} - \text{bits etiq MC}$

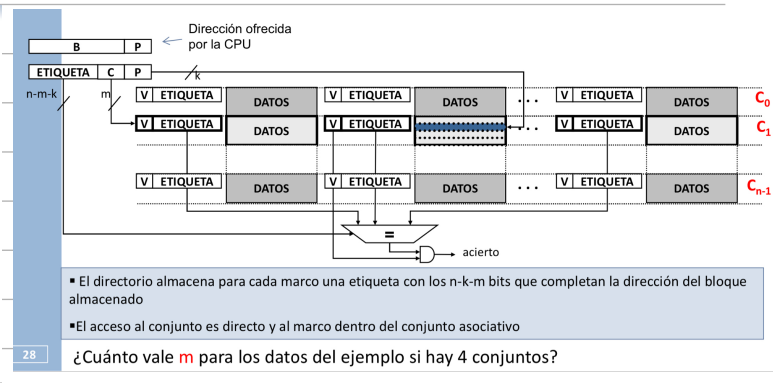
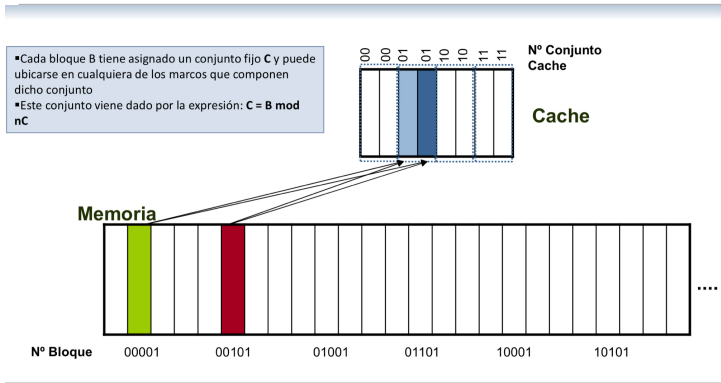
Emplat. asociativo

cada bloque B de memoria principal puede ubicarse en cualquiera de los nM bloqre de coche



Emplat. asociativo por conjuntos

Cada bloque B de memoria principal tiene asignado un conjunto fijo C y puede ubicarse en cualquier sitio



Grado de asociatividad / Número de vías

1 ⇒ Emplazamiento directo

>1 ⇒ Emplazamiento asociativo

Emplazamiento	Ventajas	Inconvenientes
Directo	Acceso simple y rápido	Alta tasa de fallos cuando varios bloques compiten por el mismo marco
Asociativo	Máximo aprovechamiento de la MC	Una alta asociatividad impacta directamente en el tiempo de acceso a la MC
Asociativo por conjuntos	Es un enfoque intermedio entre emplazamiento directo y asociativo. El grado de asociatividad afecta al rendimiento, al aumentar el grado de asociatividad disminuyen los fallos por competencia por un marco Grado más común: 2	Al aumentar el grado de asociatividad aumenta el tiempo de acceso y el coste hardware

Políticas de actualización

Escritura en memoria

write-through: todas las escrituras actualizan la cache y la memoria (al reemplazar, se puede eliminar copia de cache y solo un bit de validez)

write-back: todas las escrituras actualizan solo la cache (al reemplazar, no se pueden eliminar datos de la cache: deben ser escritos primero en la memoria de siguiente nivel y dos bits: de validez y bit de sucio)

Fallo de escritura

write allocate (con asignación en escritura): se lleva bloque que se va a escribir a la cache

no-write allocate (sin asignación en escritura): dato solo se modifica en la MP/ nivel de memoria siguiente

Políticas de reemplazamiento

Espacio de reemplazamiento conjunto de posibles bloques que pueden ser reemplazados por el nuevo bloque

- Directo (bloque que reside en el marco que el nuevo bloque tiene asignado) No se requieren algoritmos de reemplazamiento
- Asociativo cualquier bloque que resida en la cache
- Asociativo por conjuntos cualquier bloque que resida en el conjunto que el nuevo bloque tiene asignado

Algoritmos

- Aleatorio se escoge al azar **random**
- FIFO (first in, first out) se sustituye el bloque que lleve más tiempo cargado
- LRU (least recently used) se sustituye el bloque que lleve más tiempo sin haber sido referenciado
- LFU (least frequently used) se sustituye el bloque que haya sido referenciado en menos ocasiones

Rendimiento de la memoria caché

Procesador con memoria ideal

$$T_{CPU} = N * CPI * t_c$$

$$T_{CPU} = N_{\text{ciclos CPU}} * t_c$$

Procesador con memoria real

$$T_{CPU} = (N_{\text{ciclos CPU}} + N_{\text{ciclos espera memoria}}) * t_c$$

$$N * AMPI * \text{Miss Rate}$$

$$T_{CPU} = [N * CPI + (N * AMPI * \text{Miss Rate} * \text{Miss Penalty})] * t_c$$

$$T_{CPU} = N * [CPI + (AMPI * \text{Miss Rate} * \text{Miss Penalty})] * t_c$$

Tipos de fallos

- Iniciales (compulsory)

- primera referencia a un bloque que no está en cache
- depende del tamaño de bloque

- Capacidad

- si la cache no puede contener todos los bloques necesarios, hay fallos al recuperar de nuevo un bloque previamente descartado

- Conflicto

- Bloque puede ser descartado y recuperado de nuevo porque otro bloque compite por la misma línea de cache (aunque haya otras líneas libres)
- No se producen en caches puramente asociativas

Optimización de la memoria cache

$$T_{CPU} = N * [CPI + (AMPI * \text{Miss Rate} * \text{Miss Penalty})] * t_c$$

reducir

reducir tiempo de acceso
aumentar ancho de banda

Reducción de la tasa de fallos de cache (MissRate)

- Aumento tamaño del bloque

- 😊 - Disminución tasa de fallos iniciales y captura mejor localidad espacial
- 😞 - Aumento tasa de fallos de capacidad y conflicto (nº bloques $\Rightarrow$ captura peor localidad temporal)
- 😞 - Aumento penalización por fallo (Misspenalty)

- Aumento de la asociatividad

- 😊 - Disminución tasa de fallos de conflicto
- 😊 - Mayor tiempo de acierto

- Aumento del tamaño de la MC

- 😊 - Mejora tasa de fallos de capacidad
- 😞 - Puede empeorar tiempo de acceso, coste y consumo de energía

- Selección del algoritmo de reemplazamiento

- Aleatorio
- LRU

- Uso de caches víctimas memoria cache + pag. y asociativa

- Mantiene sencillos y rapidez de acceso de MC con empujamiento directo
- Contiene los bloques que han sido sustituidos más recientemente
- En un fallo primero comprueba si el bloque se encuentra en la cache víct. si es así, el bloque buscado se lleva de la cache de víctimas a la MC.
- Cuanto menor es la memoria cache más efectiva es la cache víctima